Brest State Technical University
Department of Intelligent Information Technologies

Maritime Route Planner

Application for constructing a maritime vessel's course based on neural-network classification of geographic zones

USER GUIDE

Version 1.2 (eleven sea regions, automatic coastline check of every route)
Author: Li Haoran · Supervisor: A.A. Козинский
Source code: https://github.com/EDXiaoM/MaritimeRouteAI

Brest 2026

Part I. Installation and configuration
- 1. System requirements
- 2. Installation
- 3. Preparing the data and the model
- 4. Running the application
- 5. Configuration
- 6. The database
- 7. Offline operation
- 8. Troubleshooting

Part II. Working with the application
- 1. Starting the application
- 2. Planning a route
- 3. Comparing with the shortest route
- 4. Working with AIS data
- 5. Exporting the results
- 6. Inspecting the model
- 7. Stored routes
- 8. Keyboard and mouse summary

Part I. Installation and configuration

1. System requirements

Item	Minimum	Recommended
Operating system	Windows 10 / Ubuntu 20.04 / macOS 12	any 64-bit, current
Python	3.10	3.11
RAM	4 GB	8 GB
Free disk space	3 GB	5 GB
CPU	2 cores	4 cores or more
GPU	not required	not required (CPU inference is sufficient)
Browser	any with WebSocket support	Chrome / Firefox / Edge, current

Training the model from scratch takes about five minutes on four CPU cores; no GPU is needed at any stage.

2. Installation

2.1 Obtain the sources

Clone the repository from GitHub (or unpack the delivered archive):

```
git clone https://github.com/EDXiaoM/MaritimeRouteAI.git
cd MaritimeRouteAI
```

On Windows without Git, use *Code → Download ZIP* on the GitHub page, unpack the archive and open a command prompt in the unpacked folder.

2.2 Create an isolated environment

```
python -m venv .venv
source .venv/bin/activate        # Linux / macOS
.venv\Scripts\activate           # Windows
```

2.3 Install the dependencies

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If `pip` cannot reach the default PyTorch wheels, install the CPU build explicitly:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

2.4 Verify the installation

```
pip install -r requirements-dev.txt
pytest
```

All tests must pass. Tests that need the trained model are skipped automatically when `models/zone_classifier.pt` is absent.

3. Preparing the data and the model

The archive already contains a trained model (`models/`), so this section is needed only to retrain or to use a different corpus.

The labelled corpus is attached to the GitHub release v1.0 as `merged_geojson_v2.zip` (https://github.com/EDXiaoM/MaritimeRouteAI/releases/download/v1.0/merged_geojson_v2.zip); unpack it into `data/raw/`.

```
# 1. put the labelled corpus here:
#    data/raw/merged.geojson
python scripts/prepare_dataset.py            # ~16 s, cleaning + features, writes
data/processed/
python scripts/train_model.py                # ~5 min on CPU, writes models/
python scripts/evaluate_regions.py           # optional: accuracy per sea region
python scripts/benchmark_routing.py          # optional: 6 European voyages, all methods
python scripts/regional_examples.py          # optional: 11 regional voyages + map figure
python scripts/make_figures.py               # optional: regenerate the figures
```

`prepare_dataset.py` removes the points whose class contradicts their measured height or depth. Both measurements of a point, `noaa_depth` and `elevation`, are checked; at the default tolerance of 5 m (`LABEL_CHECK_TOLERANCE_M` in `config.py`) 5 427 of the 156 838 points are removed and 151 411 remain. Pass `--no-clean` to keep them.

`prepare_dataset.py` writes:

File	Content
`data/processed/zone_dataset.npz`	feature matrices and labels for the three splits
`data/processed/dataset_statistics.json`	corpus statistics, cleaning report and per-region counts
`models/reference_index.npz`	label-free bathymetry / OSM index

`train_model.py` writes:

File	Content
`models/zone_classifier.pt`	network weights and architecture
`models/feature_scaler.joblib`	fitted standardiser
`models/model_metadata.json`	hyper-parameters, metrics and training history

4. Running the application

```
python scripts/run_server.py
```

Then open http://127.0.0.1:8000.

Options:

```
python scripts/run_server.py --host 0.0.0.0 --port 8080
python scripts/run_server.py --reload          # development: auto-reload
```

5. Configuration

All tunable values live in `src/maritime_route/config.py`. The most useful ones:

Setting	Default	Meaning
`ZONE_COST`	1.0 / 1.8 / ∞ / ∞	traversal weight of each zone
`ROUTING.grid_resolution_deg`	0.25	default lattice step, degrees
`ROUTING.max_grid_cells`	400 000	lattice budget; larger areas are coarsened
`ROUTING.safety_margin_cells`	1	width of the penalised belt along the shore
`ROUTING.proximity_penalty`	1.6	extra cost of a water cell next to land
`GENETIC.*`	see file	default population size, generations, rates and seed of the genetic algorithm
`routing/parameters.py`	see file	since version 1.3 every parameter of the lattice and of the four methods is set **in the interface** (panel *Algorithm parameters*); this file defines their defaults, ranges and help texts
`LABEL_CHECK_TOLERANCE_M`	5.0	tolerance of the data-cleaning check, metres
`REGIONS`	eleven boxes	the sea regions used for per-region evaluation
`TRAINING.*`	see file	network size, epochs, learning rate, seed
`SERVER.progress_interval_s`	0.15	minimum interval between progress frames

Several settings can also be overridden with environment variables, which is convenient for deployment:

```
export MR_HOST=0.0.0.0
export MR_PORT=8080
export MR_DB_PATH=/var/lib/maritime/routes.db
export MR_MODEL_DIR=/opt/maritime/models
export MR_OUTPUT_DIR=/var/lib/maritime/exports
export MR_RESULTS_DIR=/var/lib/maritime/results   # JSON result files (default
outputs/results)
```

6. The database

SQLite is created automatically at `data/maritime_route.db` on first start; the schema is in `src/maritime_route/storage/schema.sql`. No database server has to be installed. To reset the stored history, stop the application and delete the `data/maritime_route.db*` files.

7. Offline operation

The client requests OpenStreetMap tiles when the machine is online. If the tile service is unreachable, after four failed tiles the client switches by itself to the **on-board chart** — tiles rendered by the server from the model's own classification — and the application stays fully usable. The layer can also be selected manually in the layer control at the top-left corner of the map.

8. Troubleshooting

Symptom	Cause and remedy
`503 Model weights not found`	the model has not been trained — run `scripts/train_model.py`
The status pill shows *reconnecting…*	the server was restarted; the client reconnects by itself with exponential back-off
`Address already in use`	another process holds port 8000 — start with `--port 8081`
Planning reports *no navigable path*	departure or destination is landlocked; the lattice is already refined and widened three times automatically
Planning reports *lies on land*	the port is more than 60 km from any navigable cell, often because it lies outside the eleven sea regions covered by the training data
The genetic algorithm reports *no navigable route evolved*	the evolutionary search converged on a closed strait; use A*
A red banner *The route crosses land …*	the automatic coastline check could not correct the route (the neural network takes too much land for water there); set the grid resolution to 0.1° and build again, or check the passage on an official chart
No coastline-check banner appears	the package `global-land-mask` is not installed — `pip install -r requirements.txt`
Step limit reached before the goal	the maximum number of steps of the method is too small for this course; raise it in *Algorithm parameters* (for example ×100) or use A* instead of Dijkstra
The search takes too long	press ■ **Stop**, or set a *Time limit*; use a coarser lattice, via points, or the heuristic weight of A* above 1
Where are the result files?	`outputs/results` of the project (or `MR_RESULTS_DIR`); the path is shown after every search and on the *Data* tab
The map is blank	no internet access — switch to *On-board chart (offline)* in the layer control
`ModuleNotFoundError: maritime_route`	run the scripts from the project root, or set `PYTHONPATH=src`

Part II. Working with the application

1. Starting the application

Run `python scripts/run_server.py` and open http://127.0.0.1:8000.

The header shows two indicators:

- **live connection** — the real-time channel to the server is open. While it reads *reconnecting…* the application still works through ordinary HTTP requests, only without live progress.
- **model 96.9% acc** — the loaded classifier and its test-set accuracy.

The window is split into the control panel (left) and the chart (right). The panel has five tabs: **Plan**, **Data**, **Compare**, **Model**, **History**.

Interface language. The buttons **EN / RU / 中文** at the right of the header switch the interface between English, Russian and Chinese at any moment, also after a search: the results on the screen are redrawn in the new language. The first time, the language of the browser is used (English if it is none of the three); the choice is then remembered in the browser. Result files, PDF reports and exports are always in English, so they read the same for everybody. (Интерфейс на русском языке — кнопка **RU**; 中文界面 —— 按 **中文**。)

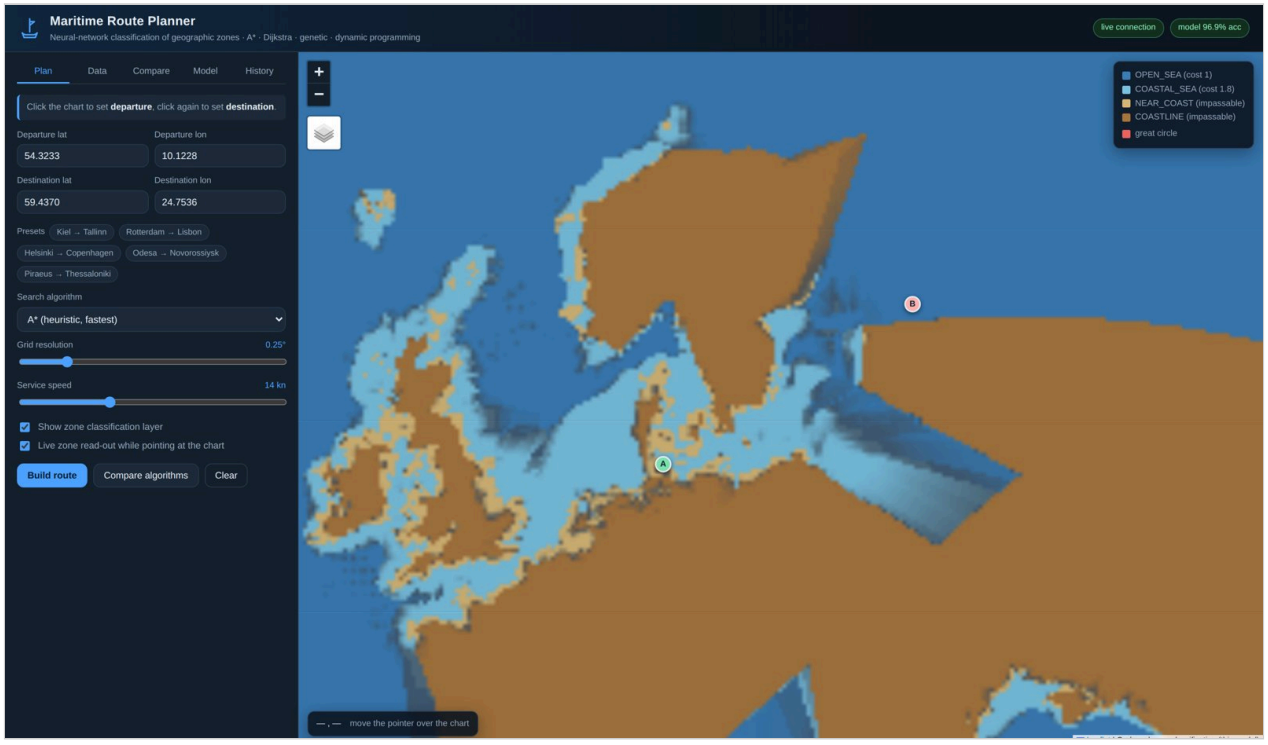


Figure 1 – Main window after start

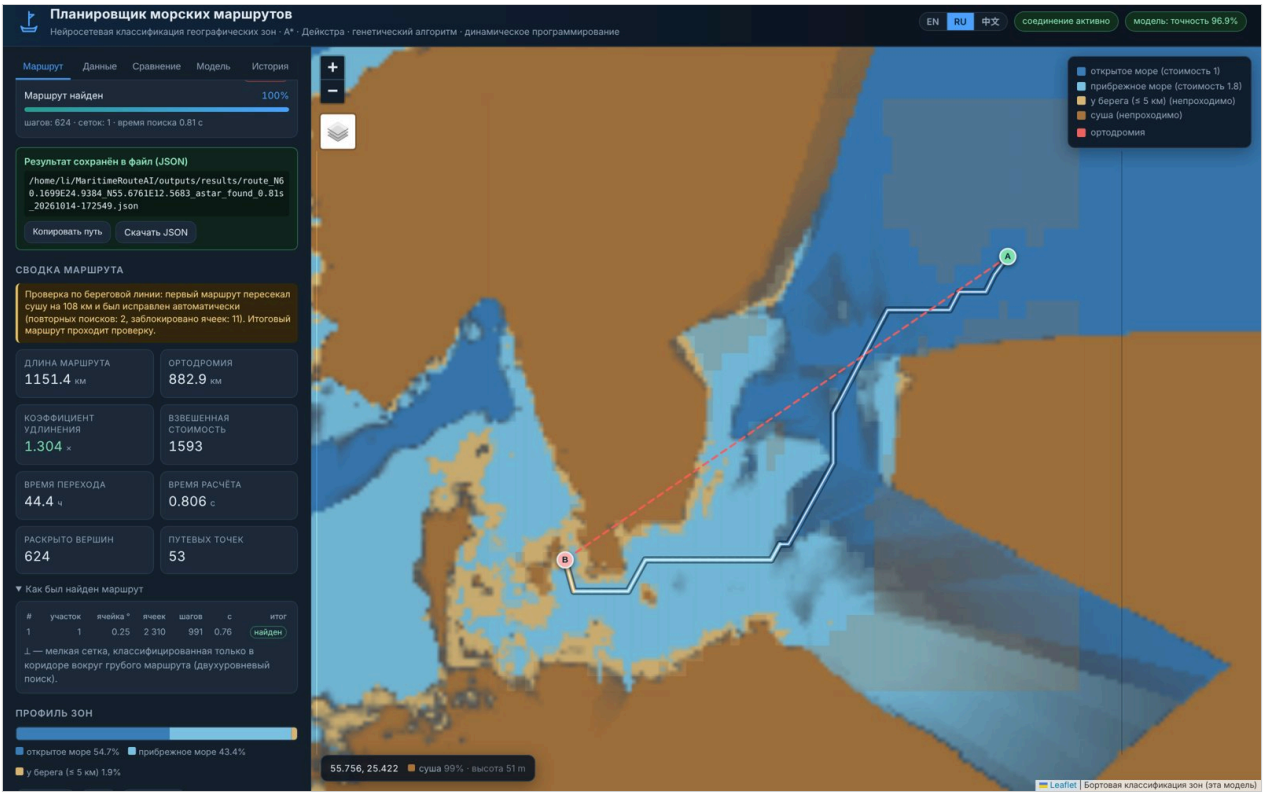


Figure 2 – The interface in Russian (button RU): a route corrected by the coastline check

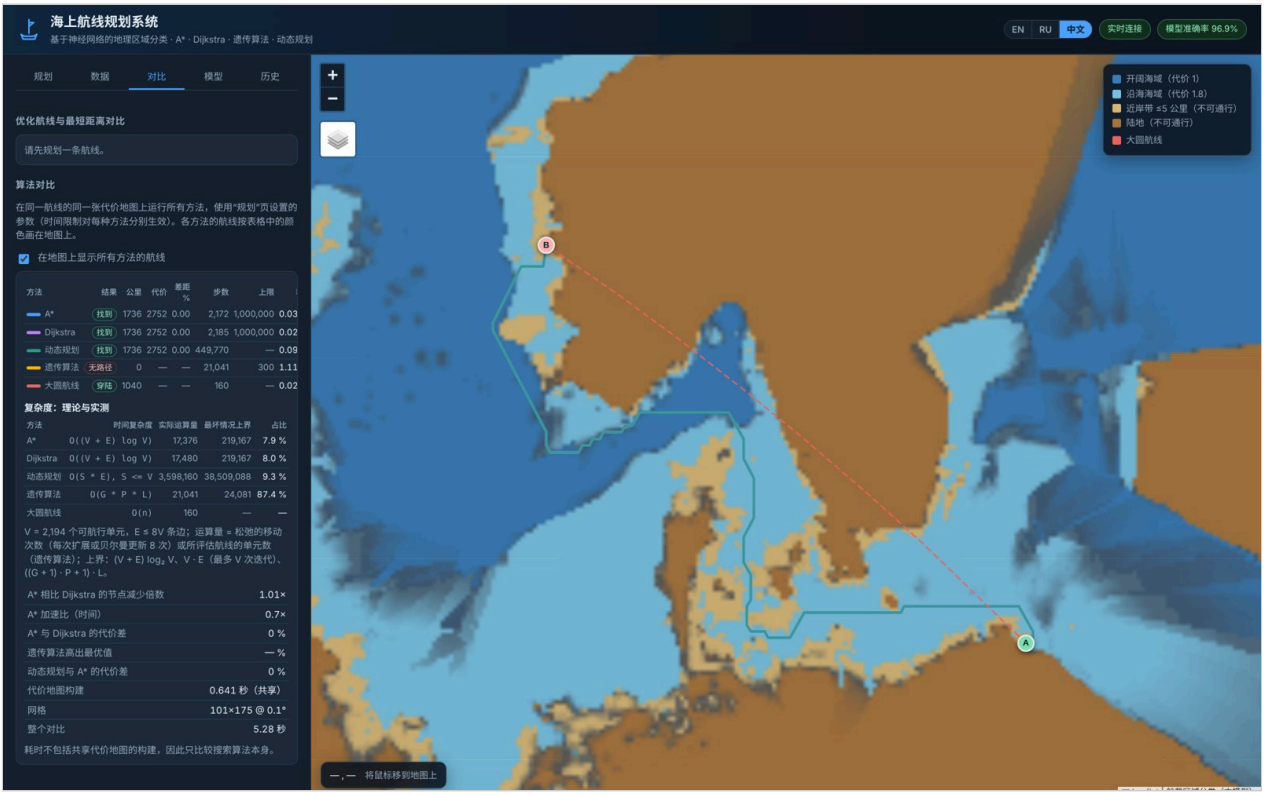


Figure 3 – The interface in Chinese (button 中文): comparison of the methods (Gdańsk → Bergen)

2. Planning a route

2.1 Setting your own route: departure, destination, via points

Any route can be set; the presets are only examples. Three equivalent ways:

- **Click the chart.** The first click sets the departure (marker **A**), the second the destination (marker **B**). The hint above the coordinate fields says which one the next click will set; the buttons **Set A on chart** and **Set B on chart** choose it explicitly, **Swap A ⇄ B** exchanges them.
- **Drag the markers.** Both markers can be dragged; the coordinate fields follow.
- **Type the coordinates** into the four fields, or press one of the **preset** chips (Kiel → Tallinn, Rotterdam → Lisbon, Helsinki → Copenhagen, Odesa → Novorossiysk, Piraeus → Thessaloniki, and the complex courses Gdańsk → Bergen, Jeddah → Dubai, Piraeus → Rhodes, Singapore → Hong Kong, Rotterdam → Gdańsk). The Piraeus presets also set the grid resolution to 0.1°, which the island-dense Aegean Sea needs; the other presets restore the default of 0.25°.

Via points (version 1.3). A complex course — round a peninsula, through a chosen strait, past a chosen side of an island group — can be forced through intermediate points: **Shift + click** on the chart (or press **Add via point** and click as many times as needed, then press it again) adds a yellow numbered marker. The route then passes through the points in their order; every leg is planned on its own lattice. Via points can be dragged, removed with a right click or with the × in the list under the coordinates, or all at once with **Remove all**.

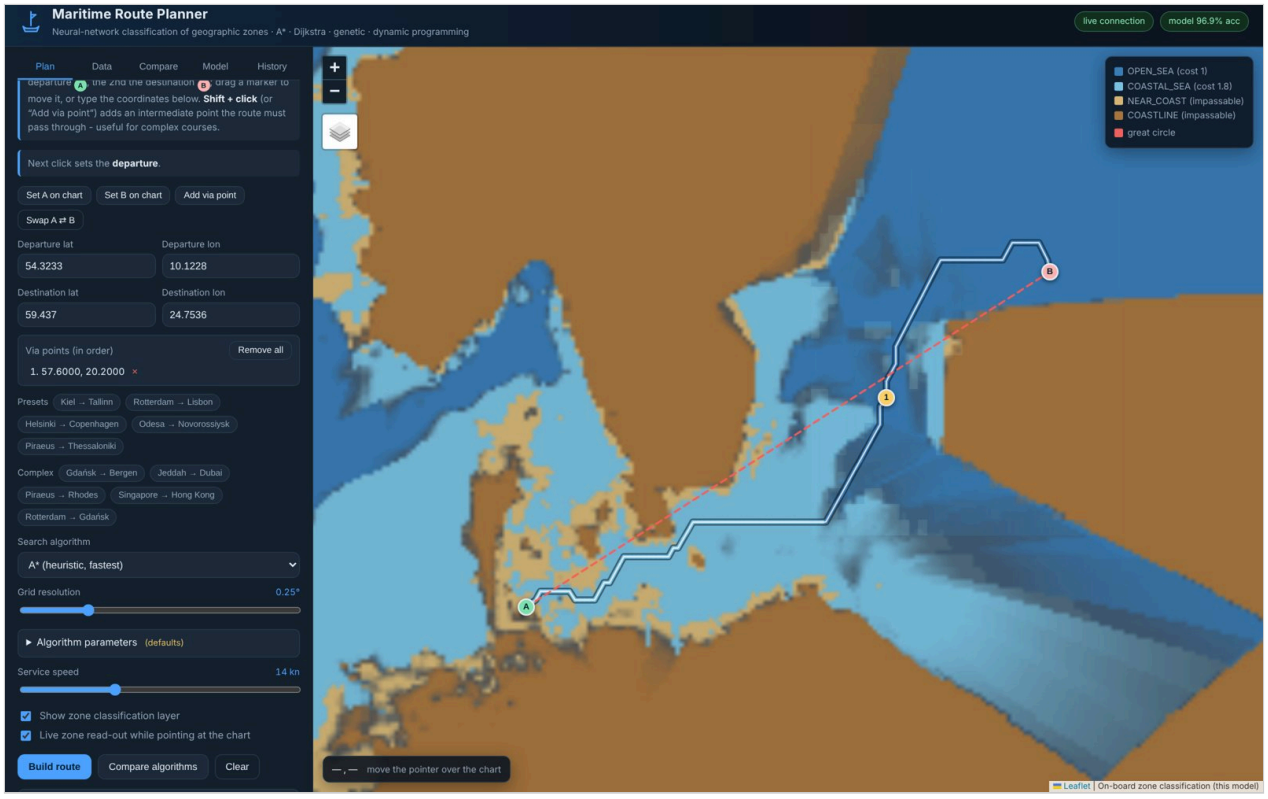


Figure 4 – Own route Kiel → Tallinn through a via point (yellow marker 1) set with Shift + click

2.2 Choosing the parameters

Control	Meaning
Search algorithm	`A*` — heuristic search, the default; `Dijkstra` — uniform cost, same result, more nodes; `Genetic algorithm` — evolutionary search: a population of candidate routes is improved by selection, crossover and mutation under a fitness function; it gives no guarantee of optimality and is shown for comparison; `Dynamic programming` — solves the Bellman equation for the cost to the destination of every cell (value iteration) and follows it from the departure; exact, same result as A*; `Great circle` — the naive shortest-distance reference

Control	Meaning
Grid resolution	lattice step in degrees (0.02–1°). Finer resolution follows narrow straits more precisely but costs more time.
Algorithm parameters	a panel with the parameters of the lattice and of the selected method (see below)
Service speed	used only to convert the route length into a passage time
Show zone classification layer	overlays the classified cells on the chart
Live zone read-out	classifies the point under the pointer continuously and shows the zone, confidence and depth at the bottom-left of the chart

Algorithm parameters (version 1.3). Every method has its own parameters, and they are set in the application, not in the code. Open the panel *Algorithm parameters* under the grid resolution: it shows the group *Lattice and run* and the group of the method selected above. Hover over a label for its explanation and default. A changed value is shown in amber and counted in the panel title; **Reset to defaults** restores the shown groups. The values are remembered in this browser.

Group	Parameter	Default	Meaning
Lattice and run	Lattice cell budget	400 000	largest lattice classified; a larger box is coarsened
	Margin around the points	2°	space around the rectangle of the points
	Diagonal moves	on	8 or 4 neighbours of a cell
	Near-shore band / penalty	1 cell / 1.6	extra cost of water next to land
	Automatic lattice refinement / maximum refinements	on / 3	finer lattice when no route exists
	Coastline check	on	check against the 1 km land mask and correct (2.5)
	Land mask in the correction / from the first lattice	on / off	block cells that the mask shows as land
	Land share that blocks a cell	0.75	share of land samples in a cell that blocks it
	Adaptive search area	on	widen the area when the passage lies outside it
	Two-level search (corridor) / half-width	on / 50 km	fine lattice only around the coarse route
	Time limit	0 = none	the search stops after this time
A*	Maximum number of steps	1 000 000	cells taken from the queue; 0 = no limit
	Heuristic weight	1	above 1: weighted A*, faster, at most that factor above the optimum
Dijkstra	Maximum number of steps	1 000 000	as for A*; Dijkstra needs several times more

Group	Parameter	Default	Meaning
Dynamic programming	Maximum number of sweeps	0 = until convergence	Bellman-equation sweeps
Genetic algorithm	Population size, Generations (epochs), Early stop after	80, 300, 60	size and length of the evolution
	Tournament size, Crossover and Mutation probability, Elite	3, 0.9, 0.35, 2	operators
	Maximum waypoints, Land penalty, Random seed	14, 200, 42	chromosome length, fitness weight of land, reproducibility

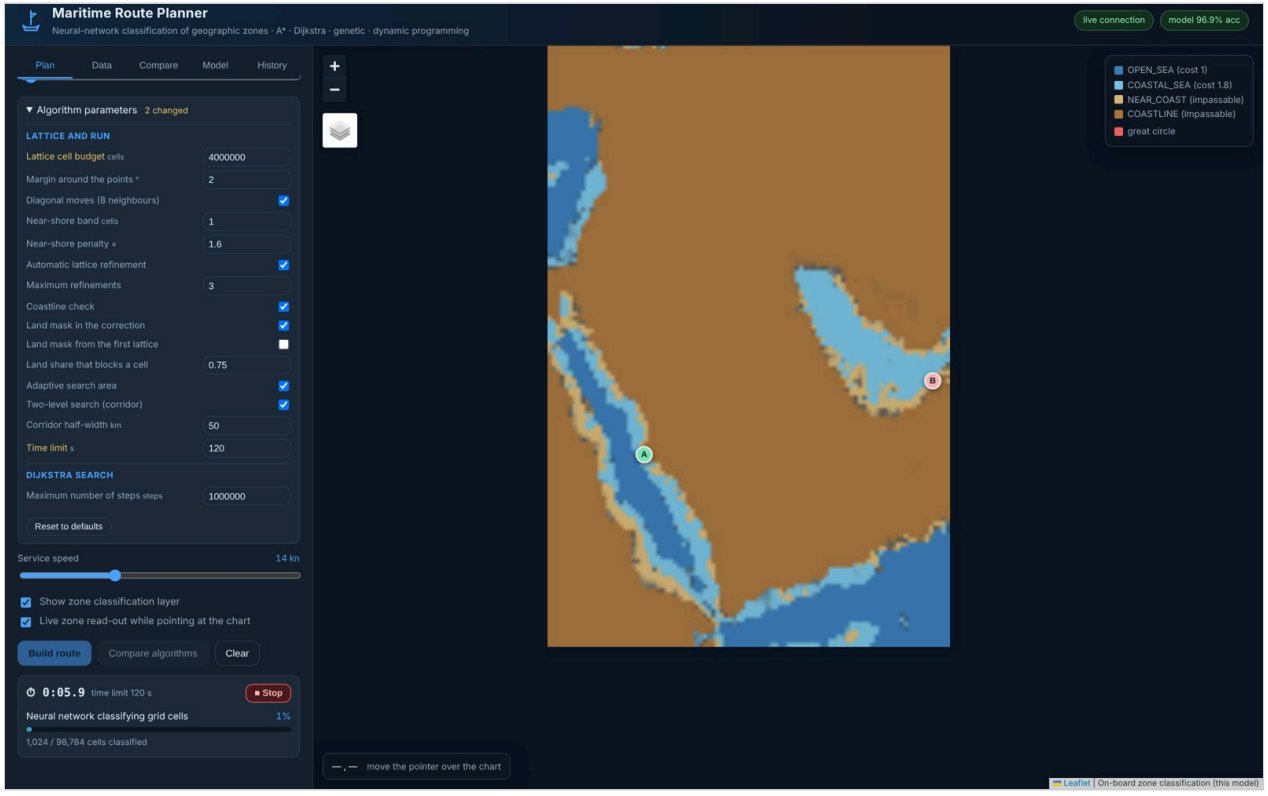


Figure 5 – Parameters of Dijkstra and of the lattice, timer and Stop button during a long search

2.3 Building the route

Press **Build route**. A progress block reports each stage:

1. *Building geographic lattice*
2. *Neural network classifying grid cells* — with the number of cells done
3. *Cost map ready* — with the share of each zone
4. *A* search in progress* — with nodes expanded and frontier size (for Dijkstra the same; for the genetic algorithm *Genetic algorithm: evolving routes* with the generation number and the best route cost so far; for dynamic programming *Dynamic programming: solving the Bellman equation* with the sweep number and the number of values improved in it)
5. *Coastline check: correcting the route* — only when the first route crosses land on the independent coastline (see 2.5)
6. *Route ready*

Timer and Stop (version 1.3). The progress block shows a timer and the time limit, if one is set. ■ **Stop** ends the search at once — within a fraction of a second, at any stage

(classification, search, correction). A search that reaches a limit ends by itself. In every case the outcome is shown and saved to a file:

Outcome	Meaning and what to do
Route found	the route is drawn
Step limit reached before the goal	the method made its maximum number of steps (A*, Dijkstra), sweeps (dynamic programming) or generations (genetic algorithm); raise it in the parameters — for example from 1 000 to 100 000 — and build again
Time limit reached	raise the time limit or use a coarser lattice
Stopped by the user	the Stop button was pressed
No navigable route between the points	the finest lattice tried has no passage; try via points, a finer lattice or a larger margin

If no route is found — because a strait is narrower than one lattice cell, or because the only passage lies outside the rectangle around the two ports — the message *Adapting the lattice* appears: when the water around both points reaches the edge of the lattice, the search area is widened (the passage lies outside it); otherwise the lattice is refined (up to three times). A departure or destination is moved onto the nearest navigable cell only if that cell is within 60 km; a point further inland is reported as lying on land.

The genetic algorithm may fail to find a navigable route where A* succeeds (it can converge on a strait that the lattice closes). In that case the lattice is not refined, because A* has proved that a route exists; choose A* instead.

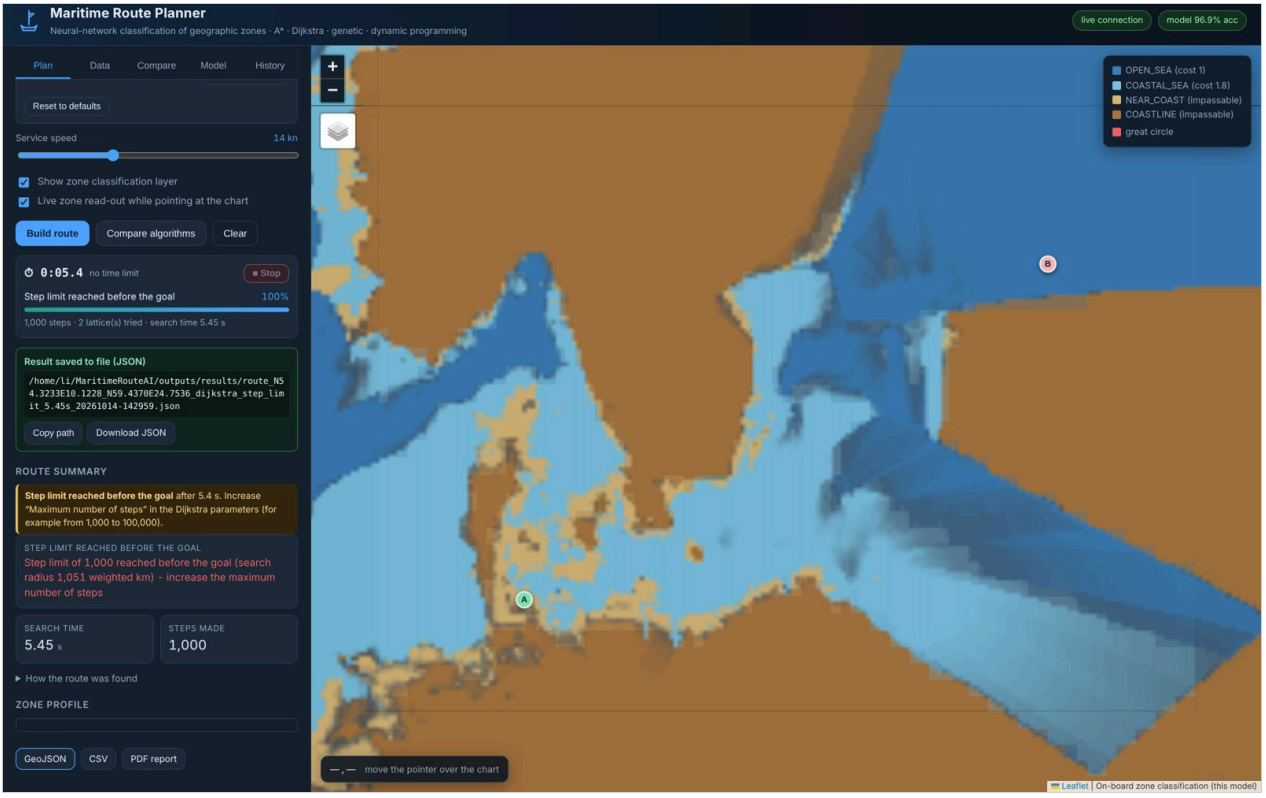


Figure 6 – A search ended by the step limit of Dijkstra (1,000 steps) and the path of its result file

2.4 Reading the result

The chart shows:

- the **optimised route**, coloured per zone along its length;
- the **great-circle route** as a red dashed line;
- the **zone classification layer**, if enabled.

The *Route summary* panel reports the route length, the great-circle distance, the detour ratio, the weighted traversal cost, the estimated passage time, the planning time, the number of expanded nodes and the number of waypoints. Below it, the *Zone profile* bar shows how much of the route lies in each zone.

How the route was found (version 1.3) opens a table of every lattice the planner tried: cell size, number of cells, steps, seconds and outcome, and what it did next (finer lattice, wider area, corridor ⊥ of the two-level search, next leg of a route with via points).

Result file (version 1.3). Above the summary a green box shows where the result was saved: the absolute path of a JSON file, a **Copy path** button and **Download JSON** (see 2.6).

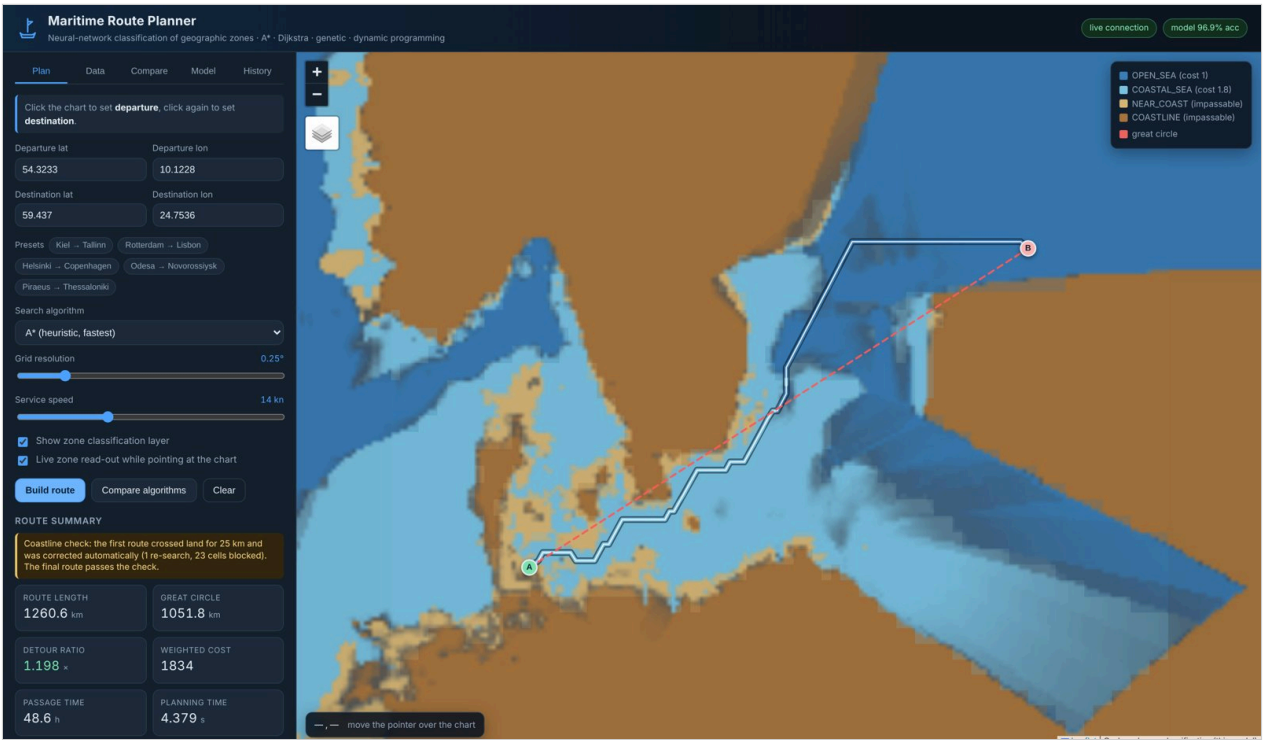


Figure 7 – Route Kiel → Tallinn with the route summary

2.5 Coastline check

Every route is checked automatically against an independent coastline — the 1 km GLOBE land mask of the package `global-land-mask`, which plays no part in the classification. The route is sampled every kilometre; the first and the last 5 km (the quays) are not checked. A banner above the route summary shows the outcome:

Banner	Meaning
green — *Coastline check passed*	no land on the route, or it only touches the coast for less than 3 km
amber — *the first route crossed land … and was corrected automatically*	the neural network had taken some land for water; the cells under that land were blocked and the route searched again (if necessary on a finer lattice); the route shown passes the check

Banner	Meaning
red — ⚠ *The route crosses land …*	the automatic correction could not remove the land; the places are marked on the chart with red circles. Do not follow such a course; try a finer grid resolution or check the passage on an official chart

A port given by a city-centre coordinate lies on land, and the straight connector from it to the open water may cross the shore: such land touching the quay is counted as the *port approach* and is not an error. The result of the check is also written to the PDF report (line *Coastline check*), to the GeoJSON export and to the database.

The check needs the package `global-land-mask` (installed with `requirements.txt`); without it the banner is not shown and the application works as before.

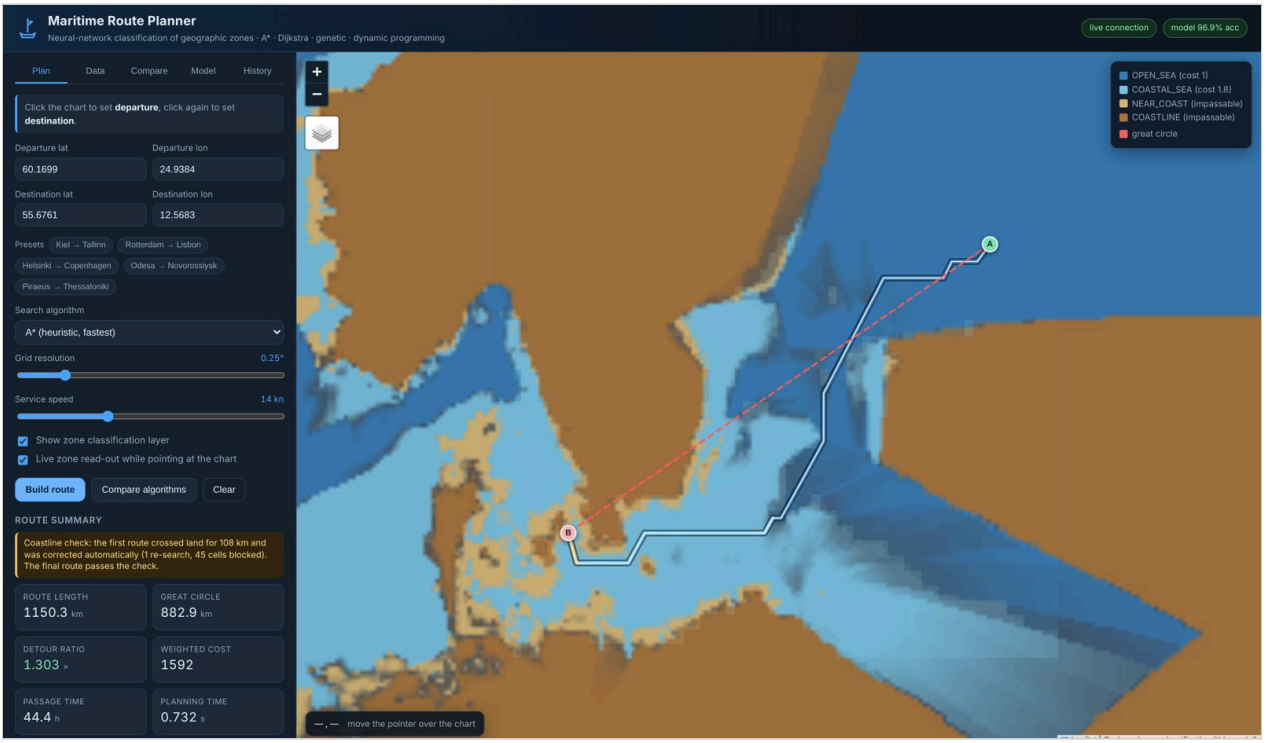


Figure 8 – A route corrected by the coastline check (preset Helsinki → Copenhagen): amber banner above the summary

2.6 Result files

Every search — found, not found, step limit, time limit or stopped — and every comparison of the methods is saved as a JSON file in the folder `outputs/results` of the project (another folder can be set with the environment variable `MR_RESULTS_DIR`). The folder is shown on the **Data** tab with the latest files. The name tells the points, the method, the outcome and the search time without opening the file:

```
route_N54.3233E10.1228_N59.4370E24.7536_dijkstra_step_limit_5.44s_20261014-134803.json
route_N54.3233E10.1228_N59.4370E24.7536_via1_astar_found_3.72s_20261014-134827.json
compare_N54.3233E10.1228_N59.4370E24.7536_5-methods_7.93s_20261014-134822.json
```

The file holds the request (points, method, resolution, speed), all parameters used, the status and its text, the timing (search seconds, start and end time), the search (steps, step limit, lattices tried), the route (length, cost, zones, coastline check) and **all points** with latitude, longitude, zone, cumulative distance and course. JSON keeps this structure in one file that

Python (`json.load`), Excel (Power Query), MATLAB (`jsondecode`) or any GIS reads without a special parser. Example in Python:

```
from maritime_route.export.results import load_result
doc, points = load_result("outputs/results/route_...json")
print(doc["status"], doc["timing"]["search_seconds"], len(points))
```

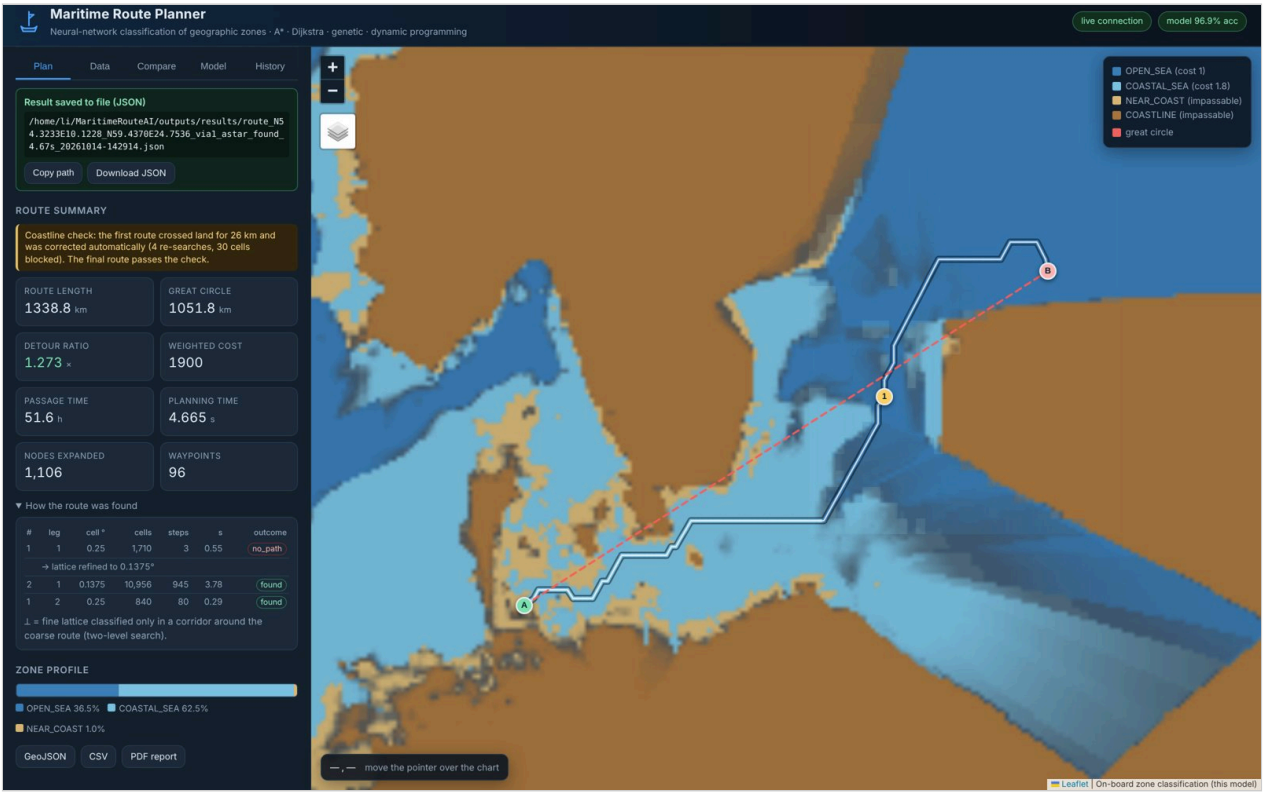


Figure 9 – Result file box and the table “How the route was found” (legs of a route with a via point)

3. Comparing with the shortest route

The **Compare** tab is filled automatically after every route. It reports the length of both routes, the additional distance accepted by the optimised route, how many sample points of the great circle fall on land, and a verdict: a great-circle route that crosses land is not navigable at all, so the extra distance is the price of a route that can actually be sailed.

Press **Compare algorithms** to run A*, Dijkstra, dynamic programming, the genetic algorithm and the great circle on one shared cost map of the same course, with the parameters set on the Plan tab (the time limit applies to each method separately; the timer and **Stop** work as for one route). The routes of all methods are drawn on the chart in the colours of the table. The first table reports for each method the outcome, route length, cost, how far above the optimum it lies, the steps made (for dynamic programming the Bellman updates, for the genetic algorithm the routes evaluated), the step limit and the search time. The second table puts the measured work next to the theoretical complexity — O((V + E) log V) for A* and Dijkstra, O(S · E) for dynamic programming, O(G · P · L) for the genetic algorithm — and its worst-case bound for this lattice. Then follow the node reduction of A* against Dijkstra, the cost differences and the size of the lattice. The reported times exclude the shared cost-map construction, so they compare the search methods only. The comparison is saved to a `compare_….json` file.

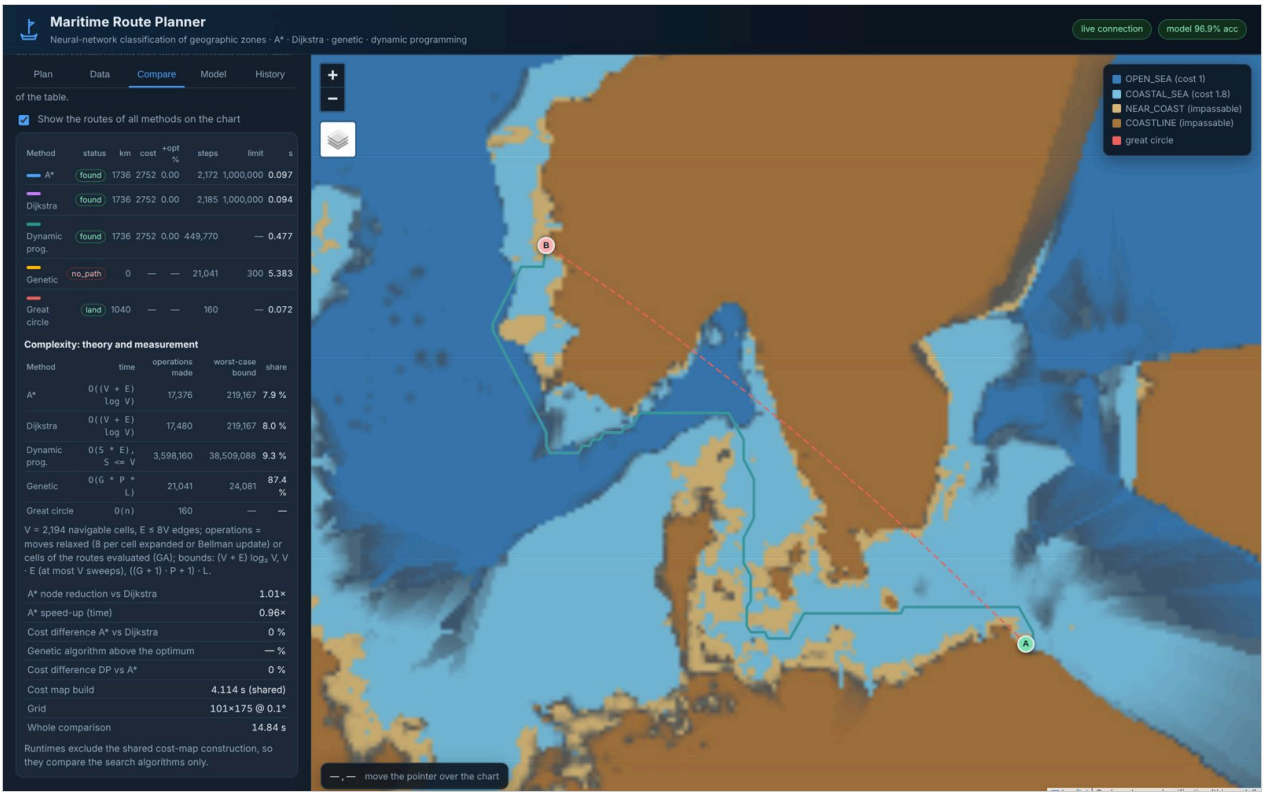


Figure 10 – Comparison of the five methods on one course (Gdańsk → Bergen): routes, outcomes, steps, complexity

4. Working with AIS data

Open the **Data** tab and drop a `.geojson`, `.csv` or `.json` file onto the drop zone (or click it to browse).

- CSV files must contain latitude and longitude columns; the usual spellings (`lat`, `lon`, `lng`, `y`, `x`, …) are recognised automatically.
- GeoJSON files must be a `FeatureCollection` of points.

The application validates the file, classifies the points and stores them in the database. The report shows how many features were read, how many were valid, how many were dropped and how the points are distributed over the four zones. If the file carries `class_code` labels, the agreement between the model and those labels is reported as well.

The classified points are drawn on the chart and can be downloaded as **Points GeoJSON** or **Points CSV**.

The *Database* block at the bottom of the tab shows how many sessions, points, routes and waypoints are stored and how large the database file is.

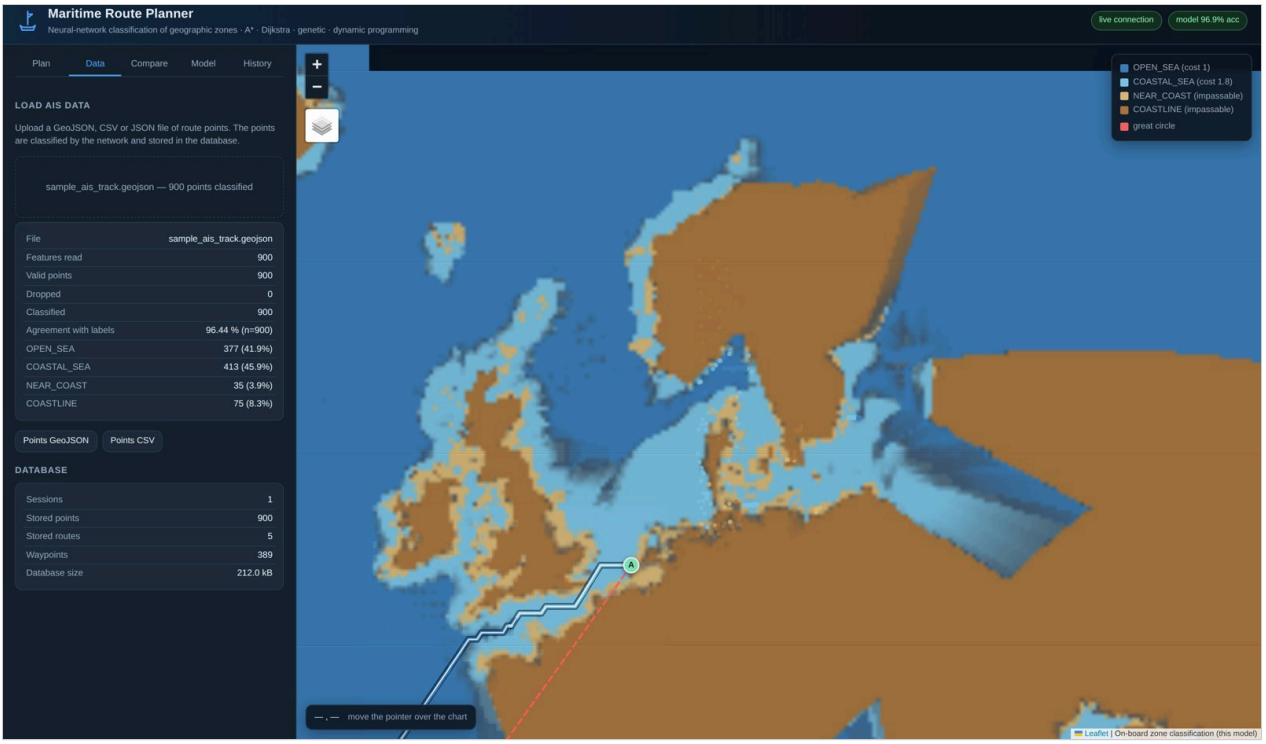


Figure 11 – Data tab after loading the sample AIS track

5. Exporting the results

After a route has been built, three buttons appear under the route summary:

Button	File	Content
GeoJSON	`route_<id>.geojson`	route line, every waypoint with its zone, and the great-circle reference line
CSV	`route_<id>.csv`	waypoint table: sequence, coordinates, zone, weight, cumulative distance, bearing
PDF report	`route_<id>.pdf`	voyage summary (including the coastline check and its warning, if any), zone profile, comparison with the shortest route, model information and the waypoint list

The GeoJSON file carries styling properties (`stroke`, `marker-color`) and opens directly in QGIS, geojson.io and similar tools.

6. Inspecting the model

The **Model** tab reports the architecture, the number of input features and parameters, the size of the reference index, and the full test-set metrics: accuracy, macro and weighted F1, Cohen's κ and macro ROC-AUC. Below them the confusion matrix is shown cell by cell, together with the training and validation curves of the run that produced the deployed weights.

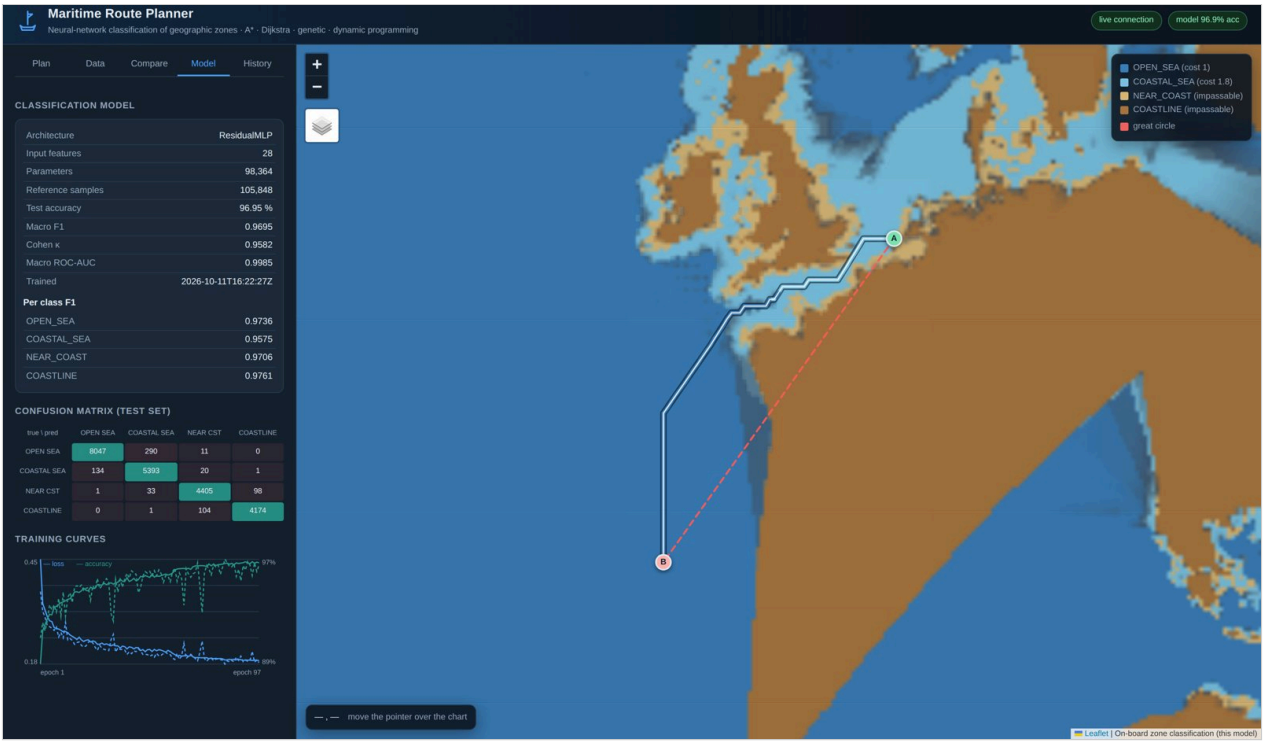


Figure 12 – Model tab: metrics, confusion matrix, training curves

7. Stored routes

The **History** tab lists the routes saved in the database, newest first, with the algorithm, length, creation time, number of waypoints, search time and detour ratio. Clicking an entry draws that route on the chart again.

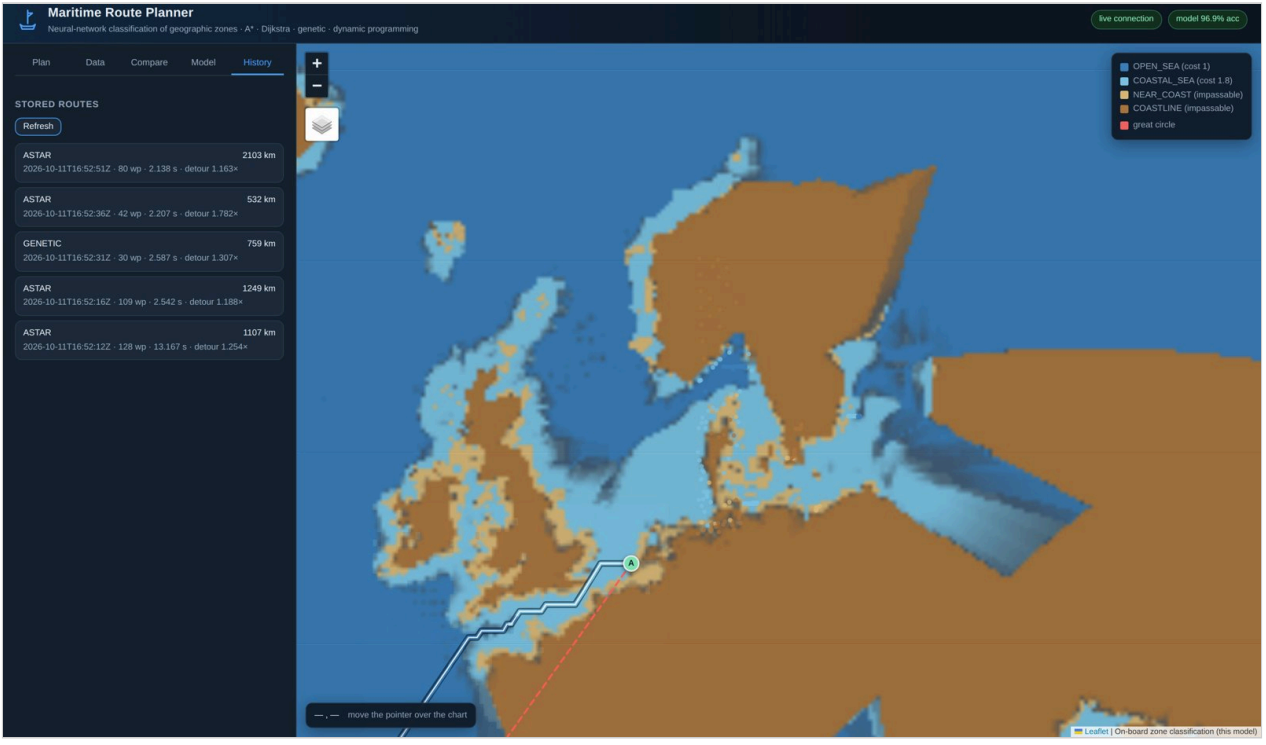


Figure 13 – History tab: routes stored in the database

8. Keyboard and mouse summary

Action	Result
Click on the chart	set departure, then destination
Shift + click on the chart	add a via point
Drag marker **A** / **B** / a via point	move it
Right click on a via point	remove it
■ **Stop** in the progress block	stop the running search
Scroll on the chart	zoom
Move the pointer over the chart	live zone read-out (when enabled)
Layer control, top-left	switch between the online chart and the on-board chart
EN / RU / 中文 in the header	switch the interface language